Change Coordination Control Module (CHCM)

Change Coordination Control Module (CHCM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Change Management Standard (SCMS)

Category: Governance & Enforcement

Subcategory: Software Change Management Governance

Type: Core Module

Governed Space: Software Change Coordination Governance

Module ID: SLGA-SCMS-CHCM-03

Origin ID: OOF-SLGA-SCMS-CHCM

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Canonical Definition

Change Coordination Control Module (CHCM) is the Core Module within Software Change Management Standard (SCMS) that governs how multiple actors, software changes, dependencies, decisions, implementation activities, operational conditions, and lifecycle governance requirements are coordinated when their relationships may affect the legitimacy, consistency, sequencing, or outcome of software modifications.

CHCM establishes change coordination as a governed relationship between

identifiable change cases and the actors, resources, dependencies, conditions, and activities required to process them without creating unmanaged conflicts or unsupported assumptions of compatibility.

The module distinguishes communication from coordination, coordination from

authorization, individual change readiness from combined change readiness, and planned implementation sequences from verified execution conditions.

It governs the identification of shared software objects, overlapping

changes, prerequisite relationships, conflicting modifications, resource constraints, coordination responsibilities, execution dependencies, decision boundaries, and material changes to coordination conditions.

CHCM recognizes that individually justified and technically valid changes may become incompatible when executed together, that implementation order can alter operational consequences, and that coordination failures may arise even when each participating actor performs its assigned task correctly.

It applies to human-operated development processes, distributed engineering

teams, continuous integration and delivery systems, cloud-native environments, automated change pipelines, AI-assisted software engineering, autonomous software agents, industrial systems, and robotic platforms.

Its principal governance output is the Software Change Coordination Profile.

Operational Role

CHCM establishes the coordination governance layer between change impact

assessment and lifecycle re-entry.

It governs:

• Change coordination identity

• Coordination scope

• Participating change cases

• Actor and responsibility references

• Shared software objects

• Interdependent modifications

• Concurrent changes

• Change sequencing

• Prerequisite conditions

• Resource and environment conflicts

• Compatibility relationships

• Decision coordination

• Authorization boundaries

• Implementation scheduling dependencies

• Integration requirements

• Coordination checkpoints

• Automated execution coordination

• Autonomous agent interactions

• Coordination evidence

• Conflict detection and resolution

• Coordination uncertainty

• Material reassessment

• Coordination continuity

• Handoff to lifecycle re-entry

CHCM does not independently authorize software implementation, replace change impact assessment, execute technical changes, or establish final change closure.

Operational Question

How can multiple software changes, actors, dependencies, decisions, and implementation activities remain coordinated when their execution order, shared resources, authority boundaries, compatibility conditions, and operational consequences may interact or change, while ensuring that conflicts, unresolved prerequisites, and coordination failures remain identifiable and governable?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CHCM is the third Core Module within SCMS.

It receives identifiable change requests from CRMM and assessed

consequences from CIAM.

It establishes coordination conditions for related changes, participating

actors, implementation dependencies, and applicable governance decisions.

Its outputs support LRECM in determining the practical coordination

requirements associated with affected lifecycle pathways.

Coordination may continue during implementation, verification, deployment, and recovery when material dependencies remain active.

• Governed Space

Software Change Coordination Governance

This governed space concerns the coordination of software change cases

whose relationships may affect implementation consistency, governance decisions, resource availability, compatibility, or operational outcomes.

It begins when a software change requires coordination with another actor, change, dependency, lifecycle activity, operational condition, or decision process.

It continues through coordination planning, dependency establishment, compatibility evaluation, sequencing, conflict management, execution alignment, reassessment, and coordination closure.

The governed space includes both planned coordination and emerging

interactions discovered during implementation or operation.

It does not replace organizational project management or the specialized governance of individual software lifecycle activities.

Governed Object

The primary Governed Object is:

A Software Change Coordination Case representing the evidence-supported

relationship between identifiable software changes, participating actors, affected software objects, shared dependencies, coordination requirements, execution sequences, applicable authority references, material conflicts, and the conditions necessary for compatible lifecycle processing.

The governed object must preserve, where applicable:

Coordination Case ID

Related Change Request IDs

Software Product References

Participating Actor References

Shared Software Objects

Dependency Relationships

Coordination Scope

Sequence Requirements

Compatibility Conditions

Resource Constraints

Authority References

Coordination Checkpoints

Conflict References

Coordination Evidence

Coordination Status

Coordination Version

Scope

CHCM governs:

Coordination Case Formation

Participating Change Identification

Actor Coordination

Responsibility Interfaces

Shared Software Object Coordination

Change Dependency Coordination

Concurrent Change Coordination

Implementation Sequence Control

Resource Coordination

Environment Coordination

Compatibility Coordination

Decision Coordination

Authority Boundary Preservation

Integration Coordination

Execution Checkpoints

Conflict Detection

Conflict Resolution

Coordination Evidence

Coordination Reassessment

Coordination Continuity

Lifecycle Re-entry Handoff

CHCM does not prescribe a particular scheduling platform, project management tool, team structure, repository strategy, or software delivery methodology.

Foundational Principle

Software changes must be coordinated according to their actual relationships and shared operational consequences, not merely according to the independent readiness of their implementation teams.

Two changes may be individually acceptable but mutually incompatible.

Two implementation actors may possess legitimate authority over separate components while lacking authority to alter a shared dependency.

A technically correct change sequence may fail when its prerequisite conditions are not established.

An automated pipeline may complete its assigned task while invalidating another active change.

Individual Change Readiness ≠ Combined Change Readiness.

Communication Completed ≠ Coordination Established.

Planned Sequence ≠ Verified Execution Sequence.

Coordination Case Identity

Every governed coordination case must have a stable identity.

Relevant elements include:

Coordination Case ID

Related Change Request IDs

Software Product References

Coordination Origin

Creation Time

Responsible Coordination Function

Coordination Scope

Current Status

Evidence Reference

Coordination Version

A coordination case may include multiple changes and revisions without losing its historical identity.

Coordination Trigger

Coordination may be required when:

Multiple Changes Affect the Same Component

Changes Share Dependencies

Implementation Activities Overlap

Release or Deployment Conditions Interact

Multiple Teams Modify Shared Interfaces

Resource Constraints Affect Execution

A Change Depends on Another Change

Conflicting Requirements Are Identified

Automated Systems Execute Related Changes

Autonomous Agents Operate on Shared Resources

Operational Conditions Require Joint Action

A coordination trigger must be associated with an identifiable relationship or material governance concern.

Coordination Purpose

Every coordination case must establish its purpose.

Potential purposes include:

Prevent Conflicting Modifications

Establish Implementation Order

Maintain Dependency Compatibility

Coordinate Shared Resources

Align Verification Activities

Preserve Authority Boundaries

Prevent Concurrent Deployment Conflicts

Coordinate Emergency Changes

Support Lifecycle Re-entry

Preserve Operational Continuity

The coordination purpose must describe the required relationship or outcome rather than merely identify a meeting or communication activity.

Coordination Scope

CHCM must define the scope of coordination.

Relevant dimensions include:

Participating Changes

Affected Software Products

Shared Components

Dependencies

Configuration

Build Artifacts

Testing Activities

Release Conditions

Deployment Environments

Operational Resources

Participating Actors

Decision Processes

Scope must reflect material interactions identified through impact assessment and subsequent evidence.

Coordination Boundary

The coordination boundary identifies the relationships and actions governed

by the coordination case.

Relevant boundaries include:

Change Case Boundary

Software Object Boundary

Actor Boundary

Authority Boundary

Resource Boundary

Environment Boundary

Execution Boundary

Temporal Boundary

External System Boundary

Operational Boundary

Coordination participation must not be treated as permission to perform actions outside an actor's legitimate authority.

Participating Change Identification

CHCM must identify all changes materially involved in the coordination case.

Relevant elements include:

Change Request ID

Impact Assessment Reference

Affected Software Objects

Proposed Modification

Change Status

Implementation Constraints

Required Lifecycle Activities

Known Dependencies

Known Conflicts

The relationship between participating changes must remain traceable.

Participating Actor Identification

CHCM must identify relevant actors and functions.

Potential participants include:

Software Developers

Software Maintainers

Quality Engineers

Security Functions

Configuration Managers

Release Managers

Deployment Operators

Infrastructure Teams

External Providers

Automated Pipelines

AI Software Agents

Autonomous Systems

Actor identity must remain distinct from assigned coordination responsibility and implementation authority.

Coordination Responsibility

CHCM must establish which functions are responsible for coordination activities.

Relevant elements include:

Coordination Function

Assigned Responsibilities

Affected Change Cases

Required Decisions

Communication Obligations

Escalation Responsibilities

Evidence Requirements

Continuity Arrangements

Coordination responsibility does not automatically establish authority to approve or execute every participating change.

Shared Software Object Identification

Multiple changes may affect the same software object.

Relevant objects include:

Source Components

Shared Libraries

Interfaces

Database Schemas

Configuration Files

Build Definitions

Runtime Services

Deployment Manifests

Automation Policies

Agent Tool Configurations

Shared-object relationships must be evaluated for compatibility, sequencing, and potential conflicts.

Change Relationship Identification

CHCM must identify material relationships among changes.

Potential relationships include:

Independent

Related

Dependent

Prerequisite

Blocking

Overlapping

Compatible

Conditionally Compatible

Conflicting

Sequence-Sensitive

Mutually Exclusive

Unknown

A relationship may change when new evidence or implementation conditions emerge.

Change Dependency Coordination

Some changes require other changes to be completed or verified first.

Relevant elements include:

Prerequisite Change

Dependent Change

Required State

Completion Evidence

Dependency Validity

Failure Condition

Alternative Pathway

Reassessment Trigger

A dependency must not be treated as satisfied solely because its associated work item is marked complete.

Implementation Sequence

CHCM must identify when implementation order materially affects the outcome.

Relevant elements include:

Required First Change

Required Subsequent Change

Intermediate Compatibility State

Prerequisite Verification

Permitted Parallel Execution

Sequence Restrictions

Failure Response

Sequence Evidence

The sequence must reflect actual dependencies and operational conditions.

Sequence Feasibility

A planned sequence must be evaluated for feasibility.

Relevant considerations include:

Artifact Availability

Dependency Compatibility

Resource Availability

Environment Readiness

Required Actors

Verification Capacity

Operational Restrictions

Recovery Capability

A theoretically valid sequence may remain infeasible under current operational conditions.

Parallel Change Execution

CHCM may permit parallel execution where material compatibility and resource conditions support it.

Relevant elements include:

Independent Change Scope

Shared Resource Exposure

Dependency Independence

Concurrent Modification Risk

Integration Requirements

Verification Requirements

Rollback Interaction

Coordination Monitoring

Parallel execution must not be justified solely by the absence of a known conflict.

Concurrent Modification Conflict

Concurrent modifications may produce conflicts.

Examples include:

Two Changes Modify the Same Interface

Two Pipelines Modify the Same Configuration

Two Agents Update a Shared Dependency

A Database Migration Conflicts With a Service Update

A Security Change Invalidates Another Change's Access Requirements

A Deployment Replaces an Artifact Required by Another Process

Material conflicts must remain identifiable and subject to appropriate resolution.

Dependency Compatibility Coordination

CHCM must coordinate compatibility conditions involving shared dependencies.

Relevant elements include:

Required Dependency Versions

Supported Version Ranges

Transitive Dependency Effects

Interface Compatibility

Runtime Compatibility

Migration Requirements

Dependency Availability

Verification Evidence

Dependency suitability remains governed by SDS.

CHCM governs the coordination relationships among affected changes.

Configuration Coordination

Multiple changes may alter shared configuration conditions.

Relevant elements include:

Configuration Baseline

Affected Parameters

Shared Configuration Objects

Environment-Specific Values

Change Sequence

Configuration Compatibility

Conflict Evidence

Restoration Requirements

Configuration coordination must preserve applicable SCFS governance references.

Build and Artifact Coordination

CHCM must identify material interactions among build and artifact activities.

Relevant elements include:

Shared Build Inputs

Artifact Dependencies

Build Sequence

Artifact Identity

Version Compatibility

Build Environment

Reproducibility Conditions

Release Candidate Dependencies

One change must not silently invalidate the artifact assumptions of another.

Quality Verification Coordination

Related changes may require coordinated verification.

Relevant elements include:

Shared Test Environments

Integration Tests

Regression Tests

Combined Behavior Tests

Test Data

Verification Sequence

Evidence Reuse

Conflicting Results

Individually successful tests do not establish combined compatibility.

Security Coordination

Changes may interact through shared security conditions.

Relevant elements include:

Authentication

Authorization

Credentials

Access Policies

Security Configuration

Dependency Exposure

Security Testing

Incident Remediation

Security-related coordination must preserve applicable SSS governance requirements.

Data Change Coordination

Changes affecting shared data structures require coordination.

Relevant elements include:

Database Schema

Data Migration

Data Producers

Data Consumers

Data Compatibility

Migration Sequence

Restoration Conditions

Data Integrity Verification

Uncoordinated data changes may create consequences that cannot be reversed through software artifact rollback alone.

Interface Coordination

CHCM must identify changes affecting shared interfaces.

Relevant elements include:

API Contracts

Message Formats

Communication Protocols

Version Compatibility

External Consumers

Integration Timing

Fallback Behavior

Deprecation Conditions

Interface coordination must account for affected parties outside the direct implementation team where applicable.

Resource Coordination

Multiple changes may depend on shared resources.

Relevant resources include:

Development Environments

Testing Infrastructure

Build Systems

Deployment Pipelines

Production Infrastructure

Database Resources

Network Capacity

Specialized Personnel

External Service Capacity

Resource availability must be assessed against concurrent obligations and operational constraints.

Environment Coordination

CHCM must preserve coordination requirements across software environments.

Potential environments include:

Development

Integration

Testing

Staging

Production

Disaster Recovery

Simulation

Physical-System Test Environment

Environment-specific compatibility and authority conditions must remain identifiable.

Release Coordination

Multiple changes may affect the formation or readiness of software releases.

Relevant elements include:

Release Candidate References

Included Changes

Excluded Changes

Dependency Compatibility

Verification Completion

Release Sequence

Release Restrictions

Release Authorization References

SRLS governs release authorization and records.

CHCM governs coordination among changes affecting those processes.

Deployment Coordination

CHCM must identify material interactions among deployment-related changes.

Relevant elements include:

Deployment Environments

Deployment Windows

Production Version Dependencies

Infrastructure Constraints

Traffic Routing

Service Continuity

Monitoring Requirements

Rollback Compatibility

Recovery Coordination

SDPS governs deployment execution and operational verification.

CHCM governs coordination dependencies among the participating changes.

Operational Continuity Coordination

Software changes may affect operational continuity.

Relevant elements include:

Service Availability

Critical Functions

Redundancy

Failover Conditions

Recovery Resources

Operational Restrictions

User Impact

External Dependencies

Coordination must account for the combined consequences of implementation activities.

Decision Coordination

Different changes may require decisions from different competent functions.

CHCM must preserve:

Required Decision

Decision Actor

Authority Reference

Affected Change

Decision Dependency

Decision Status

Decision Conditions

Evidence

Coordination of decisions does not transfer decision authority between actors.

Authority Boundary Preservation

CHCM must distinguish coordination participation from legitimate authority.

Relevant elements include:

Actor Identity

Assigned Coordination Role

Decision Authority

Implementation Authority

Permitted Software Objects

Permitted Environments

Delegation Conditions

Restrictions

Coordination Participation ≠ Implementation Authorization.

Cross-Actor Execution Boundary

Where multiple actors perform related changes, CHCM must identify boundaries between their actions.

Relevant elements include:

Assigned Tasks

Permitted Resources

Required Handoffs

Shared Dependencies

Execution Preconditions

Decision Boundaries

Intervention Rights

Escalation Requirements

An actor's authority over one component does not automatically extend to shared resources or dependent systems.

Automated Pipeline Coordination

Automated pipelines may execute related changes without direct human intervention.

CHCM must preserve:

Pipeline Identity

Assigned Change Case

Execution Scope

Shared Resources

Prerequisite Conditions

Sequence Constraints

Conflict Detection

Stop Conditions

Execution Evidence

Escalation Requirements

Automated execution must not bypass material coordination requirements merely because each pipeline is independently authorized.

Autonomous Agent Coordination

Autonomous agents may propose or execute changes affecting shared software resources.

CHCM must identify:

Agent Identity

Assigned Objective

Affected Software Objects

Permitted Tools

Shared Resources

Coordination Requirements

Execution Dependencies

Conflict Conditions

Human Override

Escalation Requirements

Multiple agents operating under individually legitimate objectives may still create incompatible outcomes.

Agent Tool-Path Coordination

A change may require an autonomous agent to use multiple tools and execution paths.

Relevant elements include:

Objective

Agent Authority

Tool Selection

Repository Access

Dependency Modification

Configuration Modification

Build Execution

Deployment Action

Evidence Collection

Resulting State

Each material tool action must remain within applicable authority and coordination boundaries.

Task Authority ≠ Constituent Action Authority.

Coordination Checkpoints

CHCM must establish checkpoints where material dependencies or conflicts require verification.

Relevant elements include:

Checkpoint ID

Required Condition

Participating Changes

Responsible Function

Evidence Requirement

Decision Reference

Permitted Continuation

Failure Response

Checkpoints may occur before implementation, during integration, before release, or before deployment.

Coordination Preconditions

A coordinated change activity may proceed only when applicable prerequisites are satisfied.

Potential prerequisites include:

Required Change Completed

Dependency Compatible

Shared Resource Available

Environment Ready

Required Verification Completed

Authority Applicable

Conflict Resolved

Recovery Capability Established

Monitoring Available

Unverified prerequisites must remain distinguishable from satisfied conditions.

Coordination Condition States

CHCM recognizes the following condition states:

Satisfied

Not Satisfied

Partially Satisfied

Unknown

Not Applicable

Exception

Each condition must retain supporting evidence or an applicable exception reference.

Unknown ≠ Satisfied.

Scheduled ≠ Ready.

Reported Complete ≠ Verified Prerequisite.

Coordination Conflict

A coordination conflict exists when material requirements, dependencies, actions, resources, or decisions cannot simultaneously be satisfied under the proposed conditions.

Examples include:

Incompatible Software Versions

Conflicting Configuration Changes

Competing Resource Requirements

Contradictory Execution Sequences

Overlapping Unauthorized Actions

Incompatible Release Conditions

Conflicting Deployment Requirements

Unresolved Authority Boundaries

A conflict must be associated with identifiable evidence and affected change cases.

Conflict Classification

CHCM may classify conflicts as:

Software Object Conflict

Dependency Conflict

Configuration Conflict

Interface Conflict

Resource Conflict

Sequence Conflict

Authority Conflict

Verification Conflict

Release Conflict

Deployment Conflict

Operational Conflict

Evidence Conflict

Multiple classifications may apply to one conflict.

Conflict Resolution

Conflict resolution may require:

Change Scope Revision

Implementation Resequencing

Dependency Adjustment

Configuration Reconciliation

Resource Reallocation

Additional Verification

Change Separation

Change Consolidation

Alternative Implementation Strategy

Escalation

Deferral

Rejection

Resolution must remain supported by the applicable governance evidence and decisions.

Conflict Escalation

Escalation is required when a conflict cannot be resolved within the applicable coordination authority or conditions.

Relevant elements include:

Conflict ID

Affected Changes

Unresolved Conditions

Required Decision

Competent Authority

Operational Consequences

Time Sensitivity

Evidence

Interim Restrictions

Escalation must not be treated as permission to continue conflicting actions.

Coordination Failure

Coordination failure occurs when required relationships or conditions cannot be maintained.

Examples include:

Prerequisite Not Established

Execution Order Violated

Shared Resource Unavailable

Conflicting Changes Executed

Required Decision Missing

Integration Verification Failed

Agent Exceeds Coordination Boundary

Operational Conditions Become Unacceptable

Coordination failure must trigger reassessment or appropriate intervention.

Coordination Uncertainty

CHCM must preserve uncertainty affecting coordination conclusions.

Examples include:

Unknown Dependency Relationship

Unverified Change Completion

Uncertain Resource Availability

Incomplete Actor Authority

Unknown Execution State

Unverified Compatibility

Missing Integration Evidence

Conflicting Status Reports

An absence of reported conflicts does not establish that coordination conditions are satisfied.

Coordination Evidence

CHCM must preserve evidence supporting material coordination conclusions.

Potential evidence includes:

Change Request References

Impact Assessment Profiles

Dependency Maps

Configuration Records

Actor Assignment Records

Authority References

Execution Plans

Checkpoint Results

Integration Test Evidence

Release and Deployment Records

Conflict Resolution Records

Operational Observations

Evidence must remain attributable to its source, relevant change cases, and observation time.

Planned–Actual Coordination Alignment

CHCM must distinguish planned coordination from actual execution conditions.

Relevant comparisons include:

Planned Participants ↔ Actual Participants

Planned Sequence ↔ Actual Sequence

Expected Dependencies ↔ Observed Dependencies

Planned Resources ↔ Available Resources

Expected Compatibility ↔ Verified Compatibility

Required Decisions ↔ Recorded Decisions

Planned Execution State ↔ Observed Execution State

Material differences must remain identifiable.

Coordination Monitoring

Coordination may require continued observation during execution.

Relevant elements include:

Change Progress

Prerequisite Completion

Resource Availability

Execution Sequence

Dependency Conditions

Conflict Indicators

Verification Results

Operational Deviations

Required Escalation

Monitoring must reflect the coordination conditions that materially affect the participating changes.

Material Coordination Change

A coordination case may require reassessment when:

A New Change Is Introduced

A Participating Change Is Withdrawn

Scope Expands

Dependencies Change

Implementation Order Changes

Resources Become Unavailable

Authority Changes

Verification Fails

Operational Conditions Change

An Autonomous Agent Introduces an Unplanned Action

Material changes must not silently inherit earlier coordination conclusions.

Coordination Reassessment

Reassessment must identify:

Previous Coordination Conditions

Changed Conditions

Affected Change Cases

Invalidated Assumptions

New Conflicts

Revised Sequence

Updated Prerequisites

Additional Decisions

Required Verification

Revised Coordination Status

Historical conclusions must remain traceable.

Coordination Continuity

CHCM must preserve coordination continuity when:

Responsible Actors Change

Change Ownership Transfers

Implementation Is Deferred

Automated Execution Replaces Manual Execution

Changes Are Consolidated or Separated

Execution Is Interrupted

Conflicts Are Reopened

Lifecycle Activities Are Repeated

Continuity requires the coordination case, participating changes,

decisions, evidence, and unresolved conditions to remain reconstructable.

Coordination Closure

Coordination closure requires an evidence-supported conclusion that the relevant coordination obligations have been resolved or appropriately transferred.

Relevant elements include:

Participating Changes

Required Coordination Conditions

Resolved Conflicts

Outstanding Dependencies

Execution Alignment

Verification References

Residual Coordination Obligations

Closure Decision

Closure Evidence

Coordination closure does not establish that every participating software change has achieved its final outcome.

Coordination Governance Classification

CHCM recognizes the following governance classifications:

Coordination Required

Coordination Case Established

Under Coordination Assessment

Dependencies Identified

Prerequisites Pending

Conflict Identified

Coordination Conditionally Established

Coordination Established

Execution Coordination Active

Coordination Deviation Identified

Coordination Paused

Reassessment Required

Evidence Insufficient

Coordination Failed

Escalation Required

Residual Obligations Outstanding

Coordination Closed

Unknown

Conflicted

Classification must remain associated with the relevant coordination case, time, scope, and supporting evidence.

Governance Requirements

A conforming CHCM implementation must satisfy the following requirements. Coordination Identity and Scope

• Establish a stable coordination case identity

• Identify participating change requests

• Preserve relevant impact assessment references

• Identify participating actors and functions

• Define coordination purpose

• Establish coordination scope and boundaries

• Identify shared software objects

• Preserve material coordination relationships

Dependencies and Compatibility

• Identify prerequisite changes

• Identify blocking dependencies

• Evaluate shared dependency compatibility

• Identify configuration interactions

• Identify interface interactions

• Evaluate concurrent change effects

• Establish sequence requirements

• Identify relevant resource constraints

• Preserve material compatibility evidence

• Identify unresolved dependency conditions

Authority and Execution

• Identify applicable coordination responsibilities

• Preserve decision authority references

• Preserve implementation authority boundaries

• Identify required execution handoffs

• Establish coordination checkpoints

• Establish execution prerequisites

• Preserve automated pipeline boundaries

• Preserve autonomous agent action boundaries

• Identify material stop or escalation conditions

• Prevent coordination participation from substituting for implementation authorization.

Conflict and Evidence

• Identify material coordination conflicts

• Classify conflicts according to their nature

• Preserve conflict evidence

• Establish appropriate resolution pathways

• Preserve escalation requirements

• Evaluate planned–actual coordination alignment

• Preserve material uncertainty

• Identify coordination failures

Continuity and Handoff

• Establish coordination monitoring where required

• Identify reassessment triggers

• Preserve material coordination revisions

• Maintain historical coordination continuity

• Establish coordination-state classification

• Preserve residual coordination obligations

• Establish evidence-supported coordination closure

• Provide structured coordination references to LRECM

• Preserve relevant references for CCLM

• Distinguish coordination completion from final change verification

Governance Logic

Change Requests → Impact Assessment References → Coordination Trigger →

Coordination Case Identity → Participating Changes → Participating Actors → Shared Software Objects → Dependency Relationships → Compatibility Assessment → Resource Conditions → Authority References → Implementation Sequence → Coordination Preconditions → Checkpoints → Conflict Assessment → Coordination Decision → Execution Alignment → Monitoring → Reassessment Where Required → Coordination Classification → Handoff / Closure

Where changes are compatible:

Related Changes → Compatibility Evidence → Sequence and Resource

Conditions → Coordination Established

Where prerequisites are unresolved:

Coordination Plan → Missing Prerequisite → Hold / Clarification /

Reassessment

Where conflicts are identified:

Conflicting Changes → Conflict Classification → Resolution Assessment →

Revised Coordination / Escalation / Deferral

Where automated systems act concurrently:

Automation A + Automation B → Shared Resource or Dependency → Execution

Boundary Assessment → Coordination Conditions → Permitted Execution / Restriction

Where actual execution differs materially from the plan:

Planned Coordination → Observed Deviation → Impact and Dependency

Reassessment → Revised Coordination Decision

CHCM must not classify coordination as established merely because participating actors have communicated or implementation schedules have been published.

Governance Outputs

The principal output is:

Software Change Coordination Profile

Supporting outputs may include:

Coordination Case Identity

Participating Change Register

Actor Coordination Register

Responsibility Interface Record

Shared Software Object Register

Change Relationship Map

Dependency Coordination Record

Compatibility Assessment Reference

Implementation Sequence Profile

Resource Coordination Record

Environment Coordination Record

Decision Coordination Register

Authority Boundary Reference

Coordination Prerequisite Register

Coordination Checkpoint Record

Conflict Register

Conflict Resolution Record

Coordination Evidence Register

Planned–Actual Alignment Record

Coordination Reassessment Record

Residual Coordination Obligation Register

Coordination Closure Record

Minimum Implementation Framework

A minimum CHCM implementation must establish eight governance layers. Layer 1 — Coordination Identity

Identify the coordination case, participating changes, responsible

functions, and coordination purpose. Layer 2 — Relationship Mapping

Identify shared software objects, dependencies, interfaces, configuration

relationships, and affected environments. Layer 3 — Compatibility and Sequence

Evaluate change compatibility, prerequisite relationships, execution order,

and permitted parallel activities. Layer 4 — Actors and Authority

Identify participating actors, responsibilities, decision references,

implementation permissions, and escalation boundaries. Layer 5 — Resources and Preconditions

Establish required resources, environment readiness, verification

conditions, coordination checkpoints, and stop conditions. Layer 6 — Conflict Governance

Identify conflicts, evaluate resolution options, preserve evidence, and

establish escalation where necessary. Layer 7 — Execution Alignment and Continuity

Compare planned coordination with actual conditions, monitor material

dependencies, and reassess changed circumstances. Layer 8 — Governed Output and Handoff

Establish the Software Change Coordination Profile, preserve unresolved

obligations, and provide structured references to LRECM and CCLM.

The implementation may be manual, software-supported, automated, or agent-assisted.

The governance requirements remain applicable regardless of the

coordination technology used.

Cross-Module Boundary

CRMM → CHCM

CRMM establishes change request identities and initial relationships.

CHCM governs the coordination of materially related changes and their

participating actors.

CIAM → CHCM

CIAM identifies impact, dependencies, interactions, and material

consequences.

CHCM translates these findings into coordination conditions, sequencing,

conflict management, and execution alignment requirements.

CHCM → LRECM

CHCM establishes coordination dependencies and conditions.

LRECM determines how affected changes must enter or re-enter applicable software lifecycle governance mechanisms.

CHCM → CCLM

CHCM preserves coordination outcomes, unresolved conflicts, and residual

obligations.

CCLM evaluates their relevance to final change disposition and closure.

Coordination Established ≠ Lifecycle Requirements Completed.

Cross-Architecture Boundary

CHCM coordinates change relationships across SLGA™ without replacing

specialized governance.

SRS — Software Requirements Standard

Requirements consistency and change-related requirements coordination.

SCS — Software Creation Standard

Development activities, software artifact creation, and implementation

dependencies.

SDS — Software Dependency Standard

Dependency compatibility and change relationships.

SCFS — Software Configuration Standard

Configuration baselines, builds, artifact identities, and configuration

conflicts.

SQS — Software Quality Standard

Testing coordination, integration verification, and quality evidence.

SSS — Software Security Standard

Security conditions, remediation dependencies, and access boundaries.

SRLS — Software Release Standard

Release candidate composition, readiness, and authorization dependencies.

SDPS — Software Deployment Standard

Deployment sequencing, production version transitions, operational

monitoring, rollback, and recovery.

SRTS — Software Retirement Standard

Migration sequencing and retirement-related change coordination.

SLCS — Software Lifecycle Control Standard

Lifecycle transitions, exceptions, and continuity.

Applicable OOF® authority, accountability, autonomous systems, and

operational reality governance references may also be required when coordination affects those domains.

Practical Applications

Enterprise Software Development

Coordinating modifications across multiple teams working on shared software

products and services.

Continuous Integration and Delivery

Preventing independently executing pipelines from introducing incompatible

artifacts, configurations, or deployment states.

Cloud-Native Systems

Coordinating changes across distributed services, shared dependencies,

databases, and infrastructure.

AI-Assisted Software Engineering

Managing multiple AI-generated modifications affecting shared repositories

and implementation assumptions.

Autonomous Software Agents

Establishing coordination boundaries when multiple agents perform related actions using shared tools and resources.

Critical Infrastructure

Preserving implementation sequencing, authority, operational continuity,

and verification across consequential software changes.

Robotics and Embedded Systems

Coordinating firmware, sensor, control logic, and operational configuration

changes across connected physical systems.

Use Case 1 — Two Independently Approved Changes Break a Shared Interface

Two software teams prepare changes to connected services.

Team A modifies an API response format.

Team B updates a dependent application using the previous response

structure.

Both teams complete their individual development and testing activities.

The changes are scheduled for deployment during the same maintenance window.

CHCM evaluates:

Change A → Change B → Shared Interface → Compatibility Conditions →

Implementation Sequence → Integration Verification

The assessment identifies:

• Both changes are individually identifiable and supported by relevant

• The changes share an interface dependency

• The new API response is incompatible with the dependent application's

• Independent testing does not establish combined compatibility

• Coordinated implementation and integration verification are required

evidence. current expectations.

The appropriate classification is:

Coordination Conflict Identified — Compatibility Resolution Required.

Individually Approved Changes ≠ Combined Operational Compatibility.

Use Case 2 — Two Autonomous Agents Modify the Same Production

Configuration

An organization operates two autonomous software maintenance agents.

Agent A is assigned to improve application performance.

Agent B is assigned to address a security configuration issue.

Both agents identify the same production configuration file as relevant to

their objectives.

Agent A modifies a runtime parameter to increase throughput.

Agent B modifies related access restrictions.

Each agent possesses permission to propose configuration changes, but

neither has unrestricted authority to apply conflicting production modifications.

CHCM evaluates:

Agent A Objective → Agent B Objective → Shared Configuration → Proposed

Actions → Authority Boundaries → Compatibility → Coordination Conditions

The assessment identifies:

• Both objectives are legitimate within their assigned scope

• The agents operate on a shared configuration object

• Their proposed modifications may interact

• Independent task authority does not establish authority for

• Coordination, compatibility assessment, and applicable authorization

unrestricted concurrent execution. are required.

The appropriate classification is:

Coordination Required — Concurrent Execution Not Established.

Individually Legitimate Agent Objectives ≠ Legitimate Uncoordinated

Execution.

Critical Distinctions

Individual Change Readiness ≠ Combined Change Readiness.

Communication Completed ≠ Coordination Established.

Planned Sequence ≠ Verified Execution Sequence.

Related Changes ≠ Compatible Changes.

Shared Schedule ≠ Established Execution Coordination.

Dependency Identified ≠ Dependency Satisfied.

Scheduled ≠ Ready.

Reported Complete ≠ Verified Prerequisite.

Unknown ≠ Satisfied.

Parallel Execution Possible ≠ Parallel Execution Acceptable.

Independent Test Success ≠ Combined Integration Verification.

Coordination Participation ≠ Implementation Authorization.

Coordination Responsibility ≠ Decision Authority.

Task Authority ≠ Constituent Action Authority.

Conflict Escalated ≠ Conflict Resolved.

Coordination Established ≠ Lifecycle Requirements Completed.

Coordination Closure ≠ Final Change Closure.

Canonical Boundary

CHCM governs the identification, coordination, compatibility, sequencing,

authority boundaries, conflict management, execution alignment, and continuity of relationships among software changes and their participating actors.

It establishes the evidence-supported conditions under which related

software modifications may proceed without unmanaged conflicts, unsupported compatibility assumptions, or ungoverned shared-resource interactions.

CHCM does not independently authorize implementation, replace impact assessment, execute software changes, or establish final software change closure.

Its principal governance output is the:

Software Change Coordination Profile

This output preserves the coordination relationships, prerequisites,

decisions, evidence, conflicts, and unresolved obligations necessary for controlled software lifecycle processing.

Canonical Closing Statement

Software changes do not become compatible merely because each change is individually justified, approved, or technically correct.

Two development teams may produce valid modifications that cannot operate together.

Two deployment pipelines may execute successfully while creating an inconsistent production state.

A dependency update may invalidate another team's implementation assumptions.

A database migration may require a sequence that independent delivery schedules fail to preserve.

An autonomous agent may perform its assigned task correctly while interfering with another agent's legitimate objective.

And a coordination plan may appear complete even though required prerequisites, authority boundaries, or compatibility conditions remain unverified.

CHCM establishes the governance structure required to distinguish these

realities.

It requires related changes to remain connected through identifiable dependencies, shared software objects, participating actors, execution sequences, and operational conditions.

It preserves the distinction between communication and coordination,

between scheduling and readiness, and between individual authority and authority over shared actions.

It ensures that material conflicts, unresolved prerequisites, resource constraints, and execution deviations remain visible and subject to appropriate resolution or escalation.

It supports automation and autonomous software engineering without allowing

parallel technical capability to substitute for coordinated governance.

And it maintains continuity as changes evolve, participants change,

dependencies shift, and implementation conditions become more complex.

The purpose of software change coordination is not merely to ensure that different actors know what others are doing. It is to establish whether their actions can legitimately and compatibly occur within the same software and operational reality.

Change Coordination Control Module (CHCM) transforms software change

coordination from administrative communication into an evidence-supported governance mechanism for compatibility, sequencing, authority alignment, and controlled lifecycle execution.